ALP USING 8086

Made By : **KASHIF SAYYAD**

MODULE I — INTRODUCTION TO 8086 MICROPROCESSOR

■ Block Diagram of a Microprocessor-based Computer System

A microprocessor-based computer system consists of four building blocks connected together by three buses — the **address bus**, the **data bus** and the **control bus**:

- **CPU (microprocessor)** — the control and processing centre; it fetches each instruction from memory, decodes it and executes it one after another.
- **Memory** — stores the program and data; it is divided into **ROM** (permanent programs, e.g. BIOS) and **RAM** (temporary working store).
- **I/O ports and peripherals** — interface cards and devices (keyboard, display, printer, disk) through which the CPU interacts with the outer world.
- **Timing and control logic** — clocks, reset and interrupt circuitry which synchronise all the units.

The address bus carries the address of a memory location or port (unidirectional), the data bus carries the actual byte or word (bidirectional), and the control bus carries the read / write, memory or I/O selection signals. The CPU repeats its fetch–decode–execute cycle by request instructions exclusively from memory.

■ 8086 Internal Architecture

The 8086 is a **16-bit microprocessor** (16-bit data bus) with a **20-bit address bus** and so can address 2^{20} = **1 MB of memory**. Its internal structure is divided into two independent working units:

- **Bus Interface Unit (BIU)** — fetches instructions ahead into the **6-byte instruction queue (prefetch queue)**, handles all the bus operations, forms physical addresses. The segment registers **CS, DS, SS and ES** and the **Instruction Pointer (IP)** belong to the BIU. A 20-bit physical address is formed as segment register × 10H + offset.
- **Execution Unit (EU)** — decodes and executes the instruction taken from the queue. It contains the **general purpose registers** AX, BX, CX, DX (each split as high / low byte: AH+AL etc.), the **pointer / index registers SP, BP, SI, DI** and the **ALU** with the **16-bit status FLAG register** (CF, PF, AF, ZF, SF, TF, IF, DF, OF).

Because the BIU keeps prefetching while the EU executes, there is a **pipelining effect** and the 8086 runs faster than earlier processors. The largest segment of memory is **64 KB**, and the whole 1 MB memory is expressed as a long list of segments, each addressed by the segment : offset pair.

■ Addressing Modes

Addressing mode is the way in which an instruction tells the processor **from where to get or where to keep the operand**. The **data addressing modes** of the 8086 are:

1. **Immediate addressing** — operand is present inside the instruction itself: MOV AX, 1234H.
2. **Register addressing** — operand is in a register: MOV AX, BX.
3. **Direct addressing** — 16-bit offset address is written in the instruction: MOV AX, [1234H] (the segment is DS).
4. **Register indirect addressing** — the operand address is held in BX, SI or DI: MOV AX, [BX].
5. **Based addressing** — effective address = base register (BX or BP) + a displacement: MOV AX, [BX+04H].
6. **Indexed addressing** — effective address = index register (SI or DI) + displacement: MOV AX, [SI+10H].
7. **Based-indexed addressing** — effective address = base + index (+ optional displacement): MOV AX, [BX+SI] or [BX+SI+06H].

Program memory addressing modes say from where the next instruction comes: **direct** (new offset written in the instruction, e.g. JMP 2000H), **indirect** (target address taken from a register or memory, e.g. JMP AX) — these are used by the transfer instructions JMP, CALL and inter-segment versions that also give a new CS.

Stack memory addressing modes use the **stack segment register SS** together with the **stack pointer SP** (and BP for based stack data). PUSH and POP fill/remove from the top of the stack; this is used for saving return addresses of CALL and for temporary saving of registers, parameters and flags.

MODULE II — INSTRUCTION SET 8086

■ Data Transfer Instructions

Data transfer instructions copy data from a source to a destination without changing any flag:

- **MOV dst, src** — copies a byte or word between register-register, register-memory, memory-register or immediate-to-register/memory. MOV does not permit memory-to-memory copy.
- **LAHF** — load AH from the lower byte of the flag register (SF, ZF, AF, PF, CF).
- **SAHF** — store the contents of AH back into the flag register.
- **XCHG reg/mem, reg** — exchanges (swaps) the contents of two registers, or of a register and memory.

The full set also includes IN and OUT (I/O ports), LEA (load effective address), LDS / LES (load pointer), PUSH, POP, XLAT, but the syllabus emphasises these four basic transfer forms. Data transfer instructions are neither arithmetic nor logical; the flags are not affected by MOV, LAHF, SAHF or XCHG.

■ Arithmetic Instructions — ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV

The arithmetic instructions perform binary arithmetic on 8-bit or 16-bit operands and set the status flags (CF, PF, AF, ZF, SF, OF) according to the result:

- **ADD dst, src** — adds the source operand to the destination operand and keeps the sum in the destination.

- **ADC dst, src** — add with carry: dst = dst + src + CF; used when adding numbers longer than one word (carry from the lower part is added into the higher part).
- **SUB dst, src** — subtracts source from destination (dst = dst − src).
- **SBB dst, src** — subtract with borrow: dst = dst − src − CF; completes multi-word subtraction.
- **INC reg/mem** — increases the operand by 1 (affects SF, ZF, PF, OF, AF but **not CF**).
- **DEC reg/mem** — decreases the operand by 1 (also leaves CF unchanged).
- **MUL src** — **unsigned** multiplication; 8-bit form: AX = AL × src; 16-bit form: (DX:AX) = AX × src. flags CF and OF are set if the upper half of the result is non-zero.
- **DIV src** — **unsigned** division; the dividend is first placed in AX (or DX:AX); 8-bit form: AL = AX / src quotient, AH = remainder; 16-bit form: AX = quotient, DX = remainder. Division by zero generates the divide interrupt.

The corresponding **signed** operations are IMUL and IDIV (and CBW / CWD extend byte to word or word to double word before signed division). For multi-byte addition/subtraction the common ALP style is ADD / ADC and SUB / SBB chains.

■ BCD and ASCII Arithmetic Instructions — DAA, DAS, AAA, AAS, AAM, AAD

Ordinary ADD / SUB work on binary numbers, but people enter decimal and ASCII digits. So the 8086 provides **adjust instructions** that convert the binary result of a previous arithmetic instruction back into decimal (BCD) or ASCII form:

- **DAA (Decimal Adjust after Addition)** — adjust AL after adding two packed BCD bytes: if the lower nibble > 9 or AF = 1, add 06H; if the upper nibble > 9 (or the result > 99H) after that, add 60H.
- **DAS (Decimal Adjust after Subtraction)** — adjusts AL after a SUB of packed BCD numbers (subtracts 06H / 60H on borrow conditions).
- **AAA (ASCII Adjust after Addition)** — after ADD of two unpacked ASCII digits, change AL into a single unpacked BCD digit (if AL > F9H add 06H and set CF / AF, AL after clearing the high nibble).
- **AAS (ASCII Adjust after Subtraction)** — converts the result of SUB of unpacked BCD into the proper unpacked BCD digit.
- **AAM (ASCII Adjust after Multiplication)** — take the product of two unpacked BCD digits in AL and divide it by 10: AH = the quotient, AL = the remainder; so AH = tens, AL = units.
- **AAD (ASCII Adjust before Division)** — the reverse of AAM: converts a two-digit unpacked BCD number in AX into plain binary in AL before DIV.

All the adjust instructions operate only on AL (or AX) and set SF, ZF, AF, PF, CF; the OF flag is left undefined. The same instruction pair code, such as ADD AL, BL followed by DAA, gives the correct **packed BCD result**.

Remember : Practical list of VSC-2/BIT/P/208: writing ALPs to add and subtract 8-bit and 16-bit numbers using different addressing modes, to multiply using MUL, to divide using DIV, success sums with

the carry, 16-bit add / subtract using the 2's complement method, and exchange of a block of data — all the programs are assembled with MASM / TASM and executed with DEBUG or the emu8086 trainer.